

Texas QSO Party Log Processor

Contest log processing system for the Texas QSO Party, sponsored by the Texas DX Society.

Adapted from Texas QSO Party software created by Charles Sanders, NO5W

Overview

What it does and produces

This software processes log files that have been captured and pre-processed by Bruce Horn's log upload system. LAQP's site hosted by Bruce is at <https://laqp.contesting.com/>

The system produces and makes available on the web two things. The first is an individual html page (at the TQP website) for each operator. This site contains the operators certificate with his/her rankings in several categories.

Second it contains an html page that contains the Final Report, showing all the leaderboards and commentary on the contest

How it does the work (high level)

There are two parts of the system:

1. The batch job is run once after all the logs are received. (It must be run in its entirety is additional logs are accepted.) More about the batch job next.
2. The web interface. This presents a page to visitors of the QSO Party when they request the Results of the contest. (see below for more information about this application)

Both parts of the system are contained in the same Github repo, since they share some functions in common.

Batch app - run once

1. First all supporting data is read into the program (e.g. county abbreviations)
2. Then all the logs are loaded into memory
3. As each log is read in, field values are validated and stored in memory
 1. the Cabrillo header fields are checked to make sure they are present and have correct values
 2. each QSO is parsed and validated and its values are stored in memory
4. Next the cross-checking of QSOs is done from the logs in memory
 1. First an index is built of all QSOs, so they can be quickly searched
 2. The index structure looks like this for the operator W5XYZ:

```
'W5XYZ': [  
  {  
    'operator': 'K5ABC',  
    'band': '20m',  
    'mode': 'PH',
```

```
        'date': '2023-04-01',  
        'time': '1430, # or 14:30`  
        'sent_qth': 'ORLN',  
        'rcvd_qth': 'MN',  
        'line_num': 42  
    }  
]
```

3. Next we iterate over each operator and each QSO, marking QSOs that are broken, have an exchange error, or are unique. We also collect aggregate statistics about failed cross checks, which are presented to the operator in his results.
4. Then a final total score is calculated for each operator
5. Next all the results are saved in the sqlite3 database. (See discussion of database below)
6. Then each of the leaderboards is generated (see discussion of Leaderboards below)
 1. A set of leaderboard data is generated for each category and saved in memory.
 2. As each leaderboard is generated, the rank of each operator in the category - based on total score - is stored in the database
7. Next the Final Report is generated. This is a single HTML file that is stored on the system and is rendered in it's entirety when requested.

Web app

It is meant to be included as a page in the Contest website.

The web app renders a query page asking the user to select either a single operator's results or the Final Report. The visitor must specify which year for the results.

For the Final Report, the app simply renders the full HTML file for the requested year.

For an individual operator, the output is created from the database and presented in a two part page. The top part is a certificate - including rankings in categories - suitable for printing on a color printer. The bottom of the page is the detailed analysis that factored into the creation of the score.

Database

The database software used is sqlite3. There is one record for each year-operator, indexed by year and callsign. The database contains the following:

- all the information from the Cabrillo log
- indications of QSOs that will not be counted in the score
- detailed stats about a log used in creating the total score and for presenting to visitors who request individual results
- error and warning messages
- ranking of the operator in every category in which he falls.

The schema is shown here:

```
CREATE TABLE IF NOT EXISTS contest_results (  
    year TEXT NOT NULL,  
    callsign TEXT NOT NULL,  
    name TEXT,  
    club TEXT,  
    exchange TEXT,  
    overlay TEXT,  
    location_type TEXT,  
    dxcc_code INTEGER,  
    dxcc_entity TEXT,  
    mode_category TEXT,  
    power_level TEXT,  
    final_score INTEGER,  
    qso_points INTEGER,  
    total_qsos INTEGER,  
    valid_qsos INTEGER,  
    total_multipliers INTEGER,  
    counties_worked TEXT,  
    counties_worked_multiplier INTEGER,  
    states_worked TEXT,  
    states_worked_multiplier INTEGER,  
    provinces_worked TEXT,  
    provinces_worked_multiplier INTEGER,  
    dx_worked TEXT,  
    dx_worked_multiplier INTEGER,  
    counties_activated TEXT,  
    rover_bonus_points INTEGER,  
    worked_n5lcc INTEGER,  
    num_n5lcc_contacts INTEGER,  
    qsos_by_band TEXT,  
    qsos_by_mode TEXT,  
    qsos_by_hour TEXT,  
    bands_worked TEXT,  
    claimed_score INTEGER,  
    errors TEXT,  
    warnings TEXT,  
    is_valid INTEGER,  
    qsos TEXT,  
    rankings TEXT,  
    created_at TEXT,  
    updated_at TEXT,  
    PRIMARY KEY (year, callsign)  
)
```

The indices created to efficiently access the data are:

```
# Create indexes for common queries  
cursor.execute('''  
    CREATE INDEX IF NOT EXISTS idx_year
```

```

        ON contest_results(year)
    '')

    cursor.execute('''
        CREATE INDEX IF NOT EXISTS idx_location_type
        ON contest_results(year, location_type)
    ''')

    cursor.execute('''
        CREATE INDEX IF NOT EXISTS idx_mode_category
        ON contest_results(year, mode_category)
    ''')

    cursor.execute('''
        CREATE INDEX IF NOT EXISTS idx_score
        ON contest_results(year, final_score DESC)
    ''')

```

Leaderboards

All information about each leaderboard is contained in the config.py file. To add, modify, or remove a leaderboard from the Final Report. Each leaderboard constitutes a set of criteria and a ranking for each operator matching that ranking.

Each leaderboard is essentially a database query. Operators are queried from the database using the criteria for a category and the operators match that query are ranked in that category.

To create the leaderboards, first a dictionary is made of short keys and the leaderboard title. This is the dictionary for the Louisiana QSO Party for 2026:

```

RANKINGS = {
    # All Operators
    'ALL': 'All Operators',

    # Non-Louisiana
    'NS': 'Class: Non Louisiana & SSB (phone)',
    'NC': 'Class: Non Louisiana & CW/Digital',
    'NM': 'Class: Non Louisiana & MIXED Modes (SSB, CW, Digital)',

    # Louisiana Fixed
    'LFA': 'Class: All Louisiana & Fixed and Rover & All Stations',
    'LFS': 'Class: Louisiana & Fixed & SSB (phone)',
    'LFC': 'Class: Louisiana & Fixed & CW/Digital',
    'LFM': 'Class: Louisiana & Fixed & MIXED Modes (SSB, CW, Digital)',

    # Louisiana Rover
    'LRA': 'Louisiana & Rover - All Stations ',
    'LRS': 'Louisiana & Rover & SSB (phone)',
    'LRC': 'Louisiana & Rover & CW/Digital',
}

```

```

'LRM': 'Louisiana & Rover & MIXED Modes (SSB, CW, Digital)',

# Overlays
# WIRES': 'WIRES Overlay'
'WA': 'WIRES Overlay - All Stations',
'WS': 'WIRES Overlay & SSB (phone)',
'WC': 'WIRES Overlay & CW/Digital',
'WM': 'WIRES Overlay & MIXED Modes (SSB, CW, Digital)',

'TA': 'TB-WIRES Overlay - All Stations',
'TS': 'TB-WIRES Overlay & SSB (phone)',
'TC': 'TB-WIRES Overlay & CW/Digital',
'TM': 'TB-WIRES Overlay & MIXED Modes (SSB, CW, Digital)',

# 'POTA': 'Parks on the Air Overlay'
'PA': 'POTA Overlay - All Stations',
'PS': 'POTA Overlay & SSB (phone)',
'PC': 'POTA Overlay & CW/Digital',
'PM': 'POTA Overlay & MIXED Modes (SSB, CW, Digital)',

# By Class
'IN': 'Inside Louisiana (Fixed or Rover)',
'OUT': 'Outside Louisiana (US, Canada, or DX)',

# By Mode combined with Power and Class
'PHQ': 'Class: Louisiana & SSB (Phone) & QRP Power',
'PHL': 'Class: Louisiana & SSB (Phone) & Low Power',
'PHH': 'Class: Louisiana & SSB (Phone) & High Power',
'CWQ': 'Class: Louisiana & CW or DIGITAL & QRP Power',
'CWL': 'Class: Louisiana & CW or DIGITAL & Low Power',
'CWH': 'Class: Louisiana & CW or DIGITAL & High Power',
'MXQ': 'Class: Louisiana & Mixed Mode & QRP Power',
'MXL': 'Class: Louisiana & Mixed Mode & Low Power',
'MXH': 'Class: Louisiana & Mixed Mode & High Power',

'PHQN': 'Class: NON-Louisiana & SSB (Phone) & QRP Power',
'PHLN': 'Class: NON-Louisiana & SSB (Phone) & Low Power',
'PHHN': 'Class: NON-Louisiana & SSB (Phone) & High Power',
'CWQN': 'Class: NON-Louisiana & CW or DIGITAL & QRP Power',
'CWLN': 'Class: NON-Louisiana & CW or DIGITAL & Low Power',
'CWHN': 'Class: NON-Louisiana & CW or DIGITAL & High Power',
'MXQN': 'Class: NON-Louisiana & Mixed Mode & QRP Power',
'MXLN': 'Class: NON-Louisiana & Mixed Mode & Low Power',
'MXHN': 'Class: NON-Louisiana & Mixed Mode & High Power',
}

```

Then for each category, the query is created. The queries are grouped into sections with a section header and a list of columns to show in that category. There is a section title, a set of fields to show as columns, and the definitions of two leaderboards, one called "IN" (state) and the other "OUT".

```
# Section 1: Class (either LA or outside of LA)
[
  {
    'section_title': 'Two Competitive Classes: Inside Louisiana (Fixed or
Rover) or Outside of Louisiana (US, Canada, DX)',
    'show': [
      ['callsign', 'CallSign'],
      ['final_score', 'Score'],
      ['name', 'Name'],
      ['exchange', 'Exchange'],
    ],
  },
  {'title': 'IN', 'ands': [['location_type in ('LA-FIXED', 'LA-ROVER')"]]},
  {'title': 'OUT', 'ands': [['location_type in ('NON-LA', 'DX')"]]},
],
```

Rather than have the programmer have to write each query in full, and to allow for a different database system to be used, a shorthand was invented to specify the query. Each category has two keys: 'title' and 'ands'. The title is used above the leaderboard table in the Final Report, but the value is not the actual title, but the key in the Rankings dictionary. So "IN" translates to an actual title of "Inside Louisiana (Fixed or Rover)".

The value of the "and" key uses SQL syntax that works with sqlite3, and has three forms:

- a list of one or more SQL queries. Examples:

```
'ands': [['location_type in ('LA-FIXED', 'LA-ROVER')"]]
'ands': [['location_type in ('NON-LA', 'DX')"], ['mode_category', 'SSB']]
'ands': [['overlay', 'TB-WIRES'], ['mode_category', 'SSB']]
```

If an element of the 'and' list is a list of just two values, e.g. ['mode_category', 'SSB'], this implies a case-insensitive equality check. If there are more than one lists in the 'ands' list, they are considered to be AND'ed together. For example, [['overlay', 'TB-WIRES'], ['mode_category', 'SSB']] means the operator must be using SSB AND the TB-WIRES overlay.

If an element uses the 'in' construction, like [['location_type in ('NON-LA', 'DX')']], this implies the location type must be EITHER 'NON-LA' OR 'DX'.

Technical Overview

The system is written in Python. The Web app uses the Flask package to manage HTML requests. It is in a repository called 'laqp' in the JeffersonARC organization on Github.com

Batch system

The batch system is run on a Linux system to product the results that are going to be used by the web system to serve the results to users visiting the contest website. Those results consist of:

- the database, either laqp.db or tqp.db. This usually includes multiple years of data.
- the final report(s) for each year, using a naming convention: final_report_.html

Web system

The web system for LAQP is currently running on the Fly.io hosting system, at <https://laqp.w5gad.org>. It runs in a DOCKER container, and there are files to create the Fly.io Docker, including fly.toml, Dockerfile. There is also a docker-compose.yml for running the web system locally. It uses wsgi for running locally, but Fly.io uses nginx (I think).

File structure for both Batch and Web

Resource files - like county abbreviations - live in the repo.

But the input log files and the output of the Batch system live elsewhere. The location of these other files can be changes in the .env or config.py files, but the default location is in a folder at the same level in the file system as the repo. It is structures like this

```
parent folder --- repo (laqp) --- reference_data (files like country
abbreviations)
    |
    --- data (laqp_data) --- database (laqp.db)
        |
        ---Final_reports --- final_report_<year>.html
            |
            --- batch_inputs (log files) --- <year> ---
*.log, .cbr
```

LAQP full static website

The repo also contains the static HTML, CSS, JS, and favicon files for the entier LAQP website. If a different contest has its own website, then the Fly.io website can just be used to render the pages in that sites "Results" section.

Converting to a different contest

Here are some guildelines for someone wishing to use this software for a contest different from the Louisiana or Texas QSO Parties.

The software uses the word "county" instead of "parish", since that is what a new contest will have. (I guess it could be province.)

There are several resource files required to run the system. Some are common to all US state QSO parties, and one is state specific:

- tx_counties.txt
- cty.plist
- dxcc_entities.csv

- states.txt
- provinces.txt

The dynamic data directory needs to be set up outside the repo, as described above.

Much of the operation of the code will be common for most state QSO Parties, especially if the logs are being collected by Bruce Horn's system. Two things that are sure to be different are the scoring and the definitions of leaderboards.

Scoring

All the scoring is done in `cross_check.py` in a function called "score_qsos" and the other functions that it calls.

Leaderboards/categories

As described above, these are defined in `config.py`. A new contest may want to use parts of what is there or remove all of it and start fresh.

Database

It will be easiest to use the `sqlite3` database, but if another database is used, it may require some changes to the `database.py` file.

Setup

Python, etc.

Make sure your system is set up to do python development, including using environments like `venv`.

Clone the repository

Start with the main branch which is what is deployed at LAQP

Install Python Dependencies

```
pip install -r requirements.txt
```

Provide reference files

County abbreviations and make sure all the other files are up-to-date

Create directory structure for log inputs and program outputs

These can be anywhere. Just need to point to them in `.env` or `config.py`

Update `.env` and `config.py`

There are many constants defined here which are used in the program.

Usage

Batch Processing (Command Line)

Here is how to process all the log files in /batch_input/. Remember that the batch app must be run with all the log files being present. If any are added or removed, the database should be deleted and the batch app run again.

```
# cd to the repo
python batch.py
```

Web Application

```
# To start a Flask development server locally
python web.py
http://0.0.0.0:5000
```

Fly.io

It's best to read their documentation if using this hosting for the Web app. There is a good GUI for creating and deploying an app. You must set up some secrets manually before you can deploy.

Contact

Louisiana QSO Party
Jefferson Amateur Radio Club
laqp@w5gad.org
Contest Manager: KJ5BYZ

Partial information - needs editing and restructuring

Key Differences: LAQP vs TQP

Categories

- **TQP:** Power (QRP/LOW/HIGH) × Mode (CWO/PHO/DGO/MIX) × Location (DX/NTX/TX-Fixed/TX-Mobile) × Operators (SO/MO)
- **LAQP:** Mode only (Phone/CW-Digital/Mixed) × Location (DX/Non-LA/LA-Fixed/LA-Rover)
 - Power is tracked but doesn't create separate categories
 - Number of operators is ignored (everyone lumped together)
 - Overlays (WIRES/TB-WIRES/POTA) are separate awards, not categories

Scoring

- **TQP:** 2 pts phone, 3 pts CW/digital
- **LAQP:** 2 pts phone, 4 pts CW/digital

Multipliers

- **TQP:** Counted once for entire contest
- **LAQP:** Counted per band AND per mode type (CW/Digital vs Phone)
 - Example: Working CADD county on 40m CW and 40m SSB = 2 multipliers

Bonuses

- **TQP:**
 - Mobile tracking: 500 pts per 5 counties worked per mobile
 - County activation: 1000 pts per county with 5+ QSOs
- **LAQP:**
 - N5LCC bonus: 100 pts one-time for working club station
 - Rover activation: 50 pts per county activated (rovers only)

Contest Period

- **TQP:** Two sessions (Saturday afternoon + Sunday afternoon)
- **LAQP:** Single session (Saturday 1400Z - Sunday 0200Z)

Development Roadmap

Phase 1: Core Processing (Current)

- ☒ Project structure
- ☒ Configuration system
- ☒ Database schema
- ☒ Log validator
- ☐ Log preparation (adapt from TQP)
- ☐ Scoring engine (adapt from TQP)
- ☐ Statistics generator (adapt from TQP)

Phase 2: Command Line Tools

- ☒ Batch processor
- ☐ Report generator
- ☐ Database utilities
- ☐ Leaderboard generator

Phase 3: Web Application

- ☐ Flask app setup
- ☐ Log upload interface
- ☐ Real-time validation
- ☐ Score lookup
- ☐ Public results page

Phase 4: Advanced Features

- ☐ Duplicate detection across logs
- ☐ Log checking (spot mismatches)
- ☐ Certificate generation
- ☐ Email notifications
- ☐ Admin dashboard

LA Rules Summary

Scoring

1. **QSO Points:** 2 for phone, 4 for CW/digital
2. **Multipliers:**
 - Non-LA: LA counties worked (per band/mode)
 - LA: counties + states + provinces + DXCC (per band/mode)
3. **Score:** QSO points × multipliers + bonuses

Categories (12 total)

- Non-LA: Phone Only, CW/Digital Only, Mixed
- LA Fixed: Phone Only, CW/Digital Only, Mixed
- LA Rover: Phone Only, CW/Digital Only, Mixed
- (Each category can have 3 power levels: QRP/Low/High)

Overlays (Separate Awards)

- WIRES: Wire antennas only
- TB-WIRES: Tribander + wires
- POTA: Parks/campgrounds/refuges

Clarifications to the LAQP Rules

- Users fill out a web form to upload the Cabrillo log file. If anything in the header section of the log file disagrees with what was entered - or is missing - the log file is rejected immediately and the user is asked to fix the log file or change the responses on the form, and resubmit. A log file may be resubmitted any number of times, with each new upload replacing the previous ones. Fields required on the form and in the Cabrillo log file are: call sign, email, power, mode, and station type. Overlay is an OPTIONAL field, but if included the values on the upload form and in the Cabrillo file must match.
- QSOs that do not match the "CATEGORY-MODE" are ignored and receive no points. For example, Phone QSOs are ignored if the mode is "CW/DIGITAL". Of course, "MIXED" mode allows any mode in QSOs.
- The operator's QTH is taken from the QSOs present, and not the "LOCATION" element in the Cabrillo file. Except for someone who has declared himself as a "Rover" all QSOs after the first must have the same QTH as the first. Any that do not meet that requirement are ignored and receive no points.
- QSOs from one non-LA operator to another are ignored and receive no points. But QSOs from one LA operator to another are valid for points and multipliers.

- The "bonus" LA call sign used is N5LCC. One OR MORE QSOs to N5LCC, regardless of band or mode, receive 100 bonus points. Contacting N5LCC multiple times does not increase bonus points. This callsign is not part of the rankings in the contest. Also it may have multiple calling QTH, and may be operating at two different frequencies on the same band as long as the mode is different.
- Warning messages are generated for all these situations so the user knows which QSOs were ignored. The log has been accepted for scoring if there are warning messages, but if the user wants to make corrections and resubmit, the last submission is used for scoring and ranking.